

1. What are the differences between Native, Hybrid, and Cross-platform mobile application development, and when is each approach commonly used?

Native development

Built separately for iOS (Swift/Objective-C) and Android (Kotlin/Java).

Pros: Best performance, full hardware access, smooth UX.

Cons: Two codebases → higher cost and longer timeline.

- **Best for:** Games, AR/VR, fintech, performance-critical apps.

Hybrid development

Web apps (HTML, CSS, JS) wrapped in a native shell.

Pros: Fastest, cheapest, one codebase.

Cons: Limited performance, less “native” feel.

- **Best for:** Simple content apps, prototypes, internal tools.

Cross-platform development

One codebase compiled to near-native apps (Flutter, React Native).

Pros: Near-native performance, faster time-to-market, lower cost.

Cons: May need native modules for advanced features.

- **Best for:** Most business apps needing iOS + Android.

2. What is the Flutter Ecosystem, and how does it help developers create modern mobile applications?

- **Flutter ecosystem** includes:
 - **Framework:** Dart language, widget system (Material & Cupertino).
 - **Engine:** Impeller/Skia for rendering every pixel directly.
 - **Hot Reload:** Instant UI updates during development.
 - **Multi-platform reach:** iOS, Android, Web, Desktop, Embedded.
 - **Integration:** Firebase, Google Maps, Ads, Payments.
 - **Community:** Thousands of packages on pub.dev.

It helps developers by enabling **fast iteration, consistent UI across platforms, and production-ready apps from a single codebase.**

3. What is the Software Development Life Cycle (SDLC), and why is it important in building successful applications?

- **SDLC** is a structured process with phases: Planning → Analysis → Design → Coding → Testing → Deployment → Maintenance.
- **Importance:**
 - Provides a clear roadmap.
 - Improves risk management and cost estimation.
 - Ensures quality through systematic testing.
 - Aligns software with business goals and user needs.

4. How do Flutter development practices and the SDLC work together in creating real-world mobile applications?

- **Integration points:**
 - **Planning & Design:** Flutter's widget system and architecture fit naturally into SDLC design phases.
 - **Development:** Hot Reload accelerates coding iterations.
 - **Testing:** Flutter supports unit, widget, and integration tests, aligning with SDLC's testing phase.
 - **Deployment:** CI/CD pipelines (GitHub Actions, Codemagic) streamline release cycles.
 - **Maintenance:** Flutter's package ecosystem and modular structure simplify updates.

Together, they ensure **efficient workflows, faster delivery, and scalable apps.**

5. Why should developers understand these concepts before starting an application project?

- **Understanding app concepts** prevents:
 - Scope creep and budget overruns.
 - Poor user adoption due to unclear goals.
 - Wrong technology choices leading to wasted effort.
- **Benefits:**
 - Clear alignment with business objectives.
 - Better market research and competitive positioning.
 - Informed choice of Native, Hybrid, or Cross-platform strategy.